COMP0005 – ALGORITHMS

Week 6 – 25 Feb 2025

Prof Licia Capra

Dept of Computer Science

OUTLINE

- Week 6 – Preview
- Week 5 – Recap
- Exercises on Search

OUTLINE

- Week 6 – Preview
- Week 5 – Recap
- Exercises on Search

WEEK 6 – PREVIEW

- Topic 3 – Search algorithms
  - 3.3 Balanced Search Tree
  - 3.4 Hash Table
- Self-practice: Applications of Search
- Lab session: TA Lab – Week (5 and) 6
- Muddiest Point (Weeks 5 & 6 - Search)

  Closes Wed 26th Feb 2025 @ 23:59

MUDDIEST POINT (SEARCH)

Question

I believe you wrote the 2024 paper so this should be used for revision. Did you write the 2019, 2020, 2021, 2022 and 2023 papers, and therefore should these be used for revision or not? If so, we dont have access to 2020 or 2021 are you able to provide them? thank you

Posted by Anonymous at Monday, 24 February 2025, 11:49 PM (11 hours 7 mins ago)

will you ever ask us to write real code rather than pseudocode? will we be marked down for writing code rather than pseudocode?

Posted by Anonymous at Saturday, 22 February 2025, 1:55 AM (3 days 9 hours ago)

Code was not provided in the videos to implement a circular array or linear array for a queue, so can we assume we dont need to know the code and just need to know how the pointers work in a linear/circular array

Posted by Anonymous at Thursday, 20 February 2025, 5:52 PM (4 days 17 hours ago)

OUTLINE

- Week 6 – Preview
- Week 5 – Recap
- Exercises on Search

WEEK 5 – RECAP

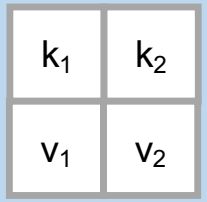
- Topic 3 – Search algorithms
  - 3.1 Symbol Table ADT

key-value pairs

`put(key, value)`
value = `get(key)`

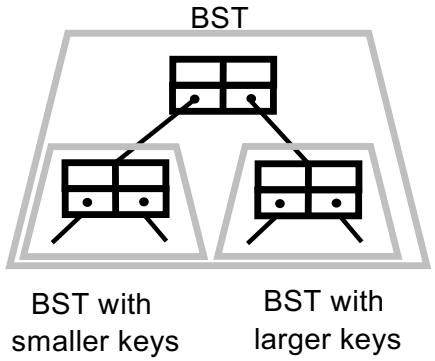
Linked List & Linear Search
→ O(N), O(N)

Ordered Arrays & Binary Search
→O(N), O(log N)



WEEK 5 – RECAP

- Topic 3 – Search algorithms
  - 3.2 Binary Search Tree (inc. advanced operations)



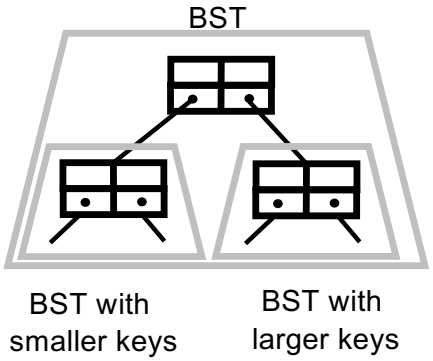
Insert keys: 9, 8, 2, 6, 1, 5, 4, 7

Insert keys: 1, 2, 4, 5, 6, 7, 8, 9

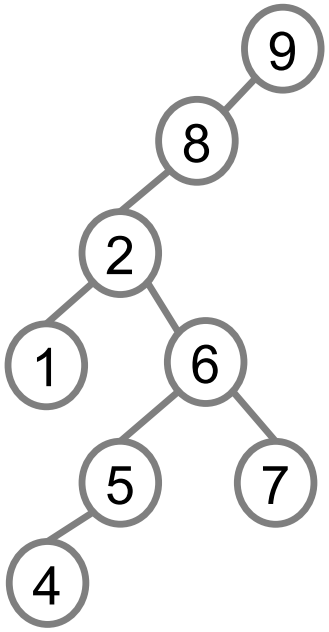
Insert keys: 6, 4, 8, 7, 9, 2, 5, 1

WEEK 5 – RECAP

- Topic 3 – Search algorithms
  - 3.2 Binary Search Tree (inc. advanced operations)



Insert keys: 9, 8, 2, 6, 1, 5, 4, 7



Average Case
O(log N), O(log N)

Worst Case
O(N), O(N)

OUTLINE

- Week 6 – Preview
- Week 5 – Recap
- Exercises on Search
  - #1 – Understanding
  - #2 – Problem solving

EXERCISES ON SEARCH

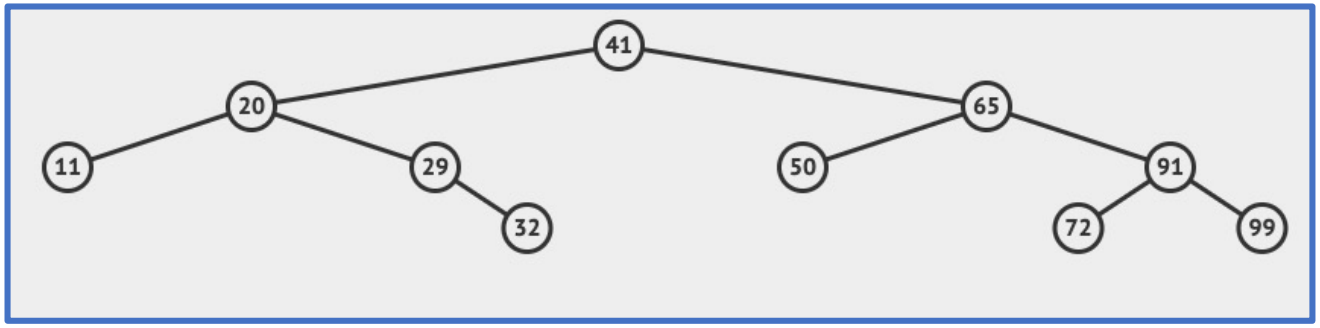
Consider the following sequence of keys:

X V T R P N

1) Illustrate the content of a Binary Search Tree (BST) after inserting all keys above in the given order [6 marks]
2) Illustrate the content of a Left Leaning Red Black (LLRB) binary search tree after inserting all keys above in the given order [15 marks]

EXERCISES ON SEARCH

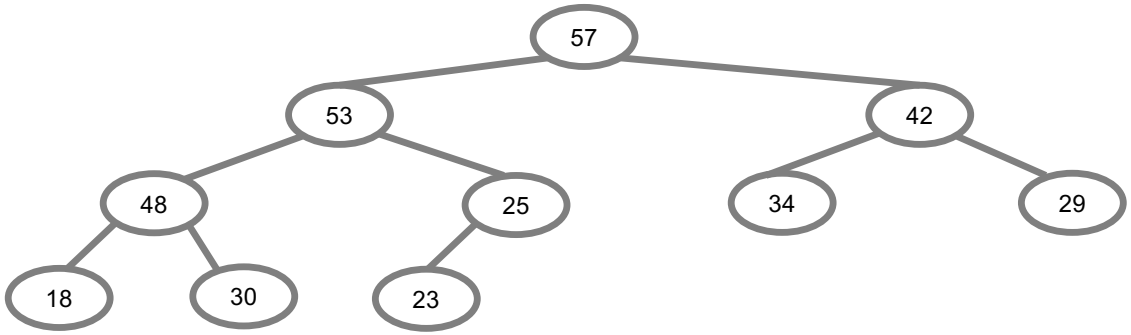
Given the following Binary Search Tree:



Illustrate its content after performing this sequence of operations: `insert(35)`, `insert(68)`, `insert(50)`, `delete(29)`, `delete(65)`, `delete(20)`, **`insert(75)`**, **`delete(68)`**

EXERCISES ON SEARCH

Consider the following tree-like representation of a max-heap:



a) Show an array-like representation of the above max-heap. Leave the content of the array at index 0 empty (as seen in lectures) and start your max-heap at position 1 in the array [4 marks]

b) …

c) …

d) Given the following array, state whether it represents a max-heap or not. Fully justify your answer [9 marks]

-	82	60	30	8	71	25	29	5	6	17	1	12

OUTLINE

- Week 6 – Preview
- Week 5 – Recap
- Exercises on Search
  - #1 – Understanding
  - #2 – Problem solving

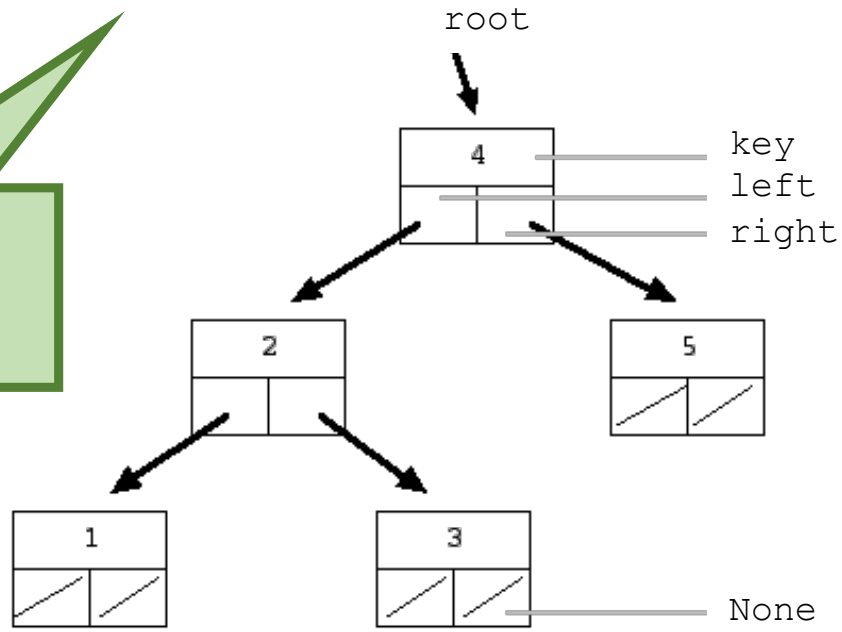
EXERCISES ON SEARCH

Write a recursive function `treeToList(root)` that takes in input a binary search tree and rearranges the internal pointers to make a circular doubly linked list out of the tree nodes. The list should be arranged so that the node keys are in **increasing order**.

EXERCISES ON SEARCH

Binary Search Tree

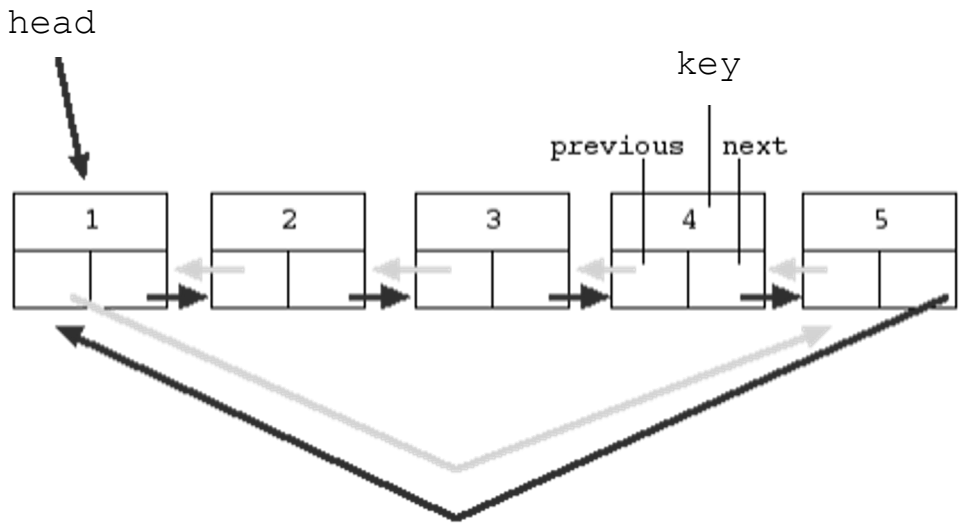
NON-LINEAR DATA STRUCTURE



Credits: Nick Parlante, Stanford University

EXERCISES ON SEARCH

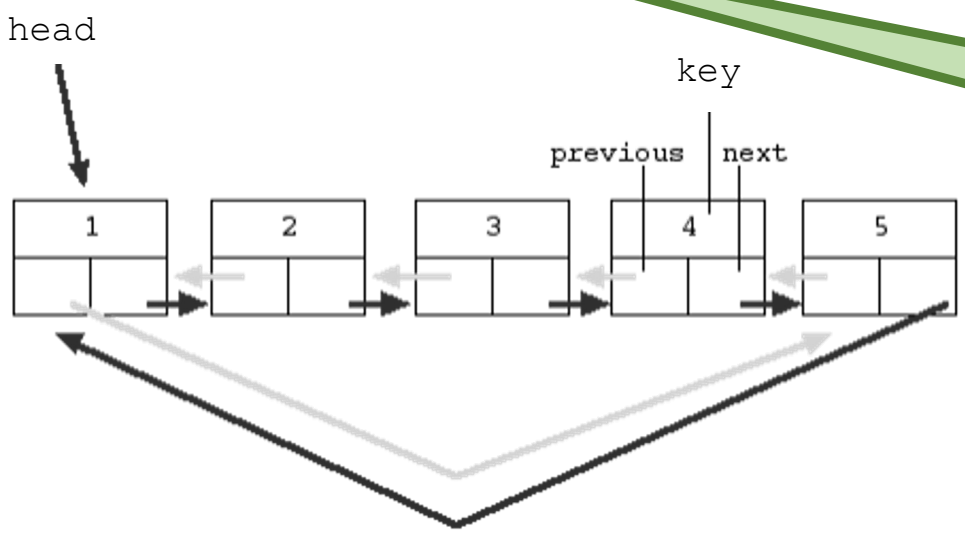
Circular doubly-linked list



Credits: Nick Parlante, Stanford University

EXERCISES ON SEARCH

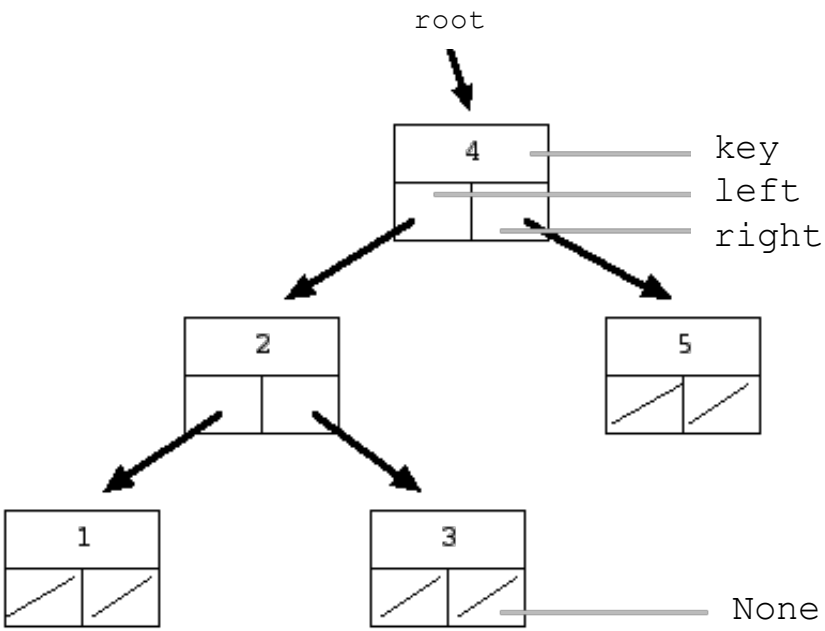
Circular doubly-linked list [ORDERED]

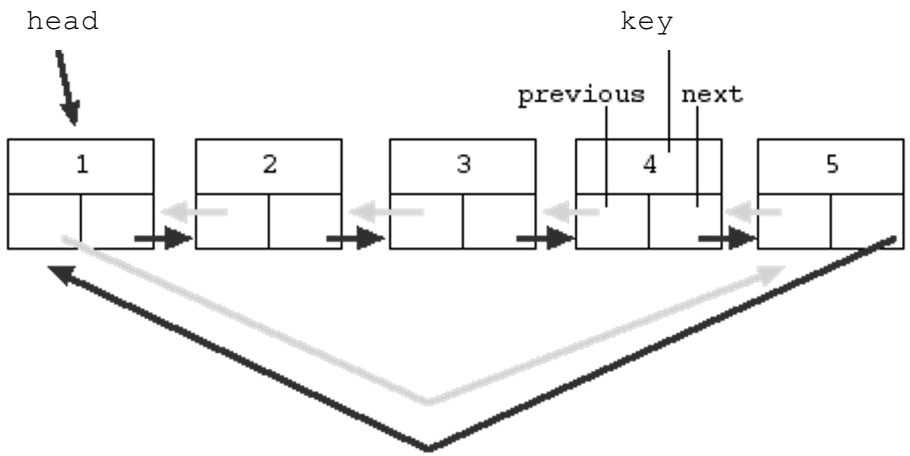


LINEAR DATA STRUCTURE

Credits: Nick Parlante, Stanford University

EXERCISES ON SEARCH





Credits: Nick Parlante, Stanford University

EXERCISES ON SEARCH

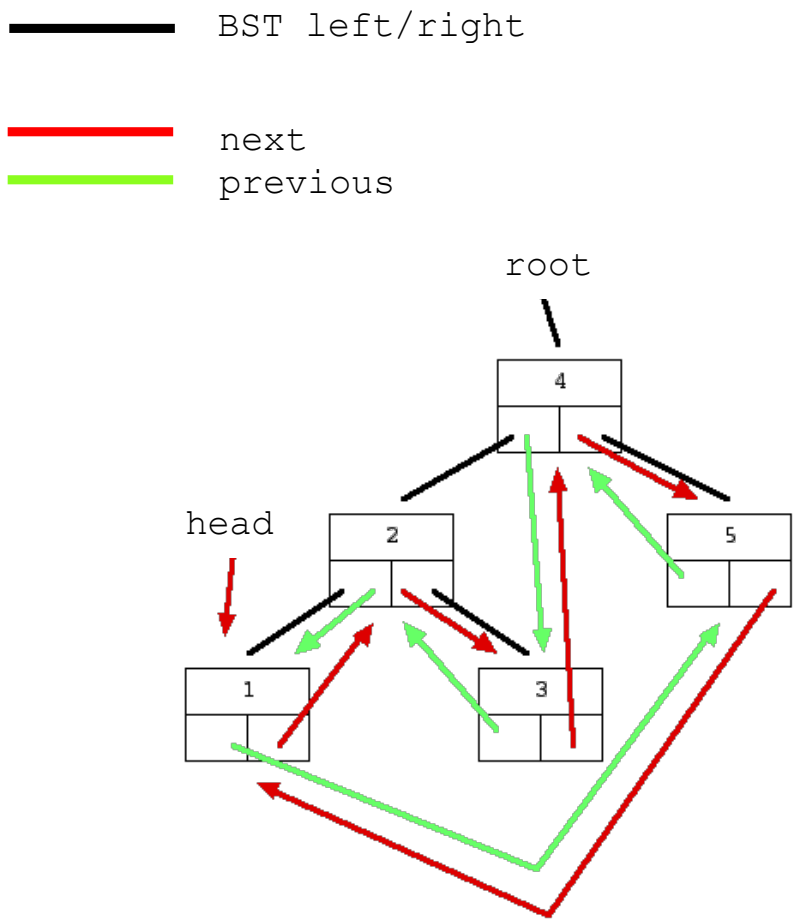
Write a recursive function `treeToList(root)` that takes in input a binary search tree and rearranges the internal pointers to make a circular doubly linked list out of the tree nodes. The list should be arranged so that the node keys are in **increasing order**.

EXERCISES ON SEARCH

Write a recursive function `treeToList(root)` that takes in input a binary search tree and rearranges the internal pointers to make a circular doubly linked list out of the tree nodes. The list should be arranged so that the node keys are in increasing order. The list `previous` pointer should be stored in the `left` field of the tree node, and the list `next` pointer should be stored in the `right` field of the tree node. Return the `head` pointer to the new list.

EXERCISES ON SEARCH

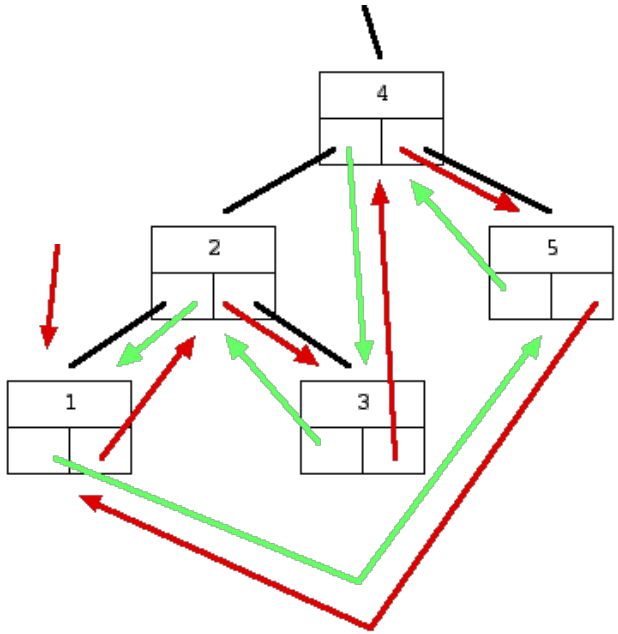
Challenge: take a BST in input and rearrange the internal pointers to make a circular doubly linked list out of it in *O*(N).



Credits: Nick Parlante, Stanford University

EXERCISES ON SEARCH

- Download skeleton code from Moodle (under week 6): `week6-exe.ipynb`
  - `Node` data type already defined
  - Instrumentation code given
  - Your task: implement `treeToList(root)` → `head` recursive function

EXERCISES ON SEARCH

- Hint: at each step, the recursion will go down the tree, recursively changing the left/right sub-trees into lists, and then append those lists together with the parent node to make a longer list.
  - Use a separate utility function `append(a, b)` that takes two circular doubly linked lists (with heads `a` and `b`) and appends them together to make one list whose head is returned.